Maestro - AI-first higher education

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Handling success and failure with .then() / .catch()

Today

Welcome to your next lesson: Handling success and failure with .then() / .catch().

I'm preparing your personalized lesson using what you've learned so far, what's coming next, and how you learn best.

Step by step, you’re moving forward.

Let’s start with the success path of fetch() using .then() 🎯

Imagine this code:

Javascript

```
console.log("Starting request...");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("Request succeeded!");
  });

console.log("Request sent.");
```

You don’t need to type anything yet — just look at it.

console.log("Starting request..."); runs immediately.
fetch(...) starts a network request and returns a Promise.
.then(() => { ... }) tells JavaScript: “when the request successfully finishes, run this callback.”

So .then() here is our success handler ☑

We’re not reading the response body at all, just logging that it worked.

Ron, does this idea make sense so far:

.then() = “run this when the fetch Promise is fulfilled (success)”?

If yes, I’ll have you write and run a tiny version of this in the editor next.

Yes, that makes sense

Nice 👍

Next step: let’s have you use .then() with fetch() yourself.

In the code editor, write and run this (type it yourself, don’t paste):

Javascript
```
console.log("A: before fetch");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("B: fetch succeeded");
  });

console.log("C: after fetch call");
```
Run it (you can use Run first), then Submit code.

Don’t tell me the output yet — just get it running.

After you submit, I’ll ask you about the order of the logs.

Javascript
```
console.log("A: before fetch");

fetch("https://jsonplaceholder.typicode.com/posts/1")
    .then(() => {
        console.log("B: fetch succeeded");
      });

console.log("C: after fetch call");
```
Nice work getting that running 👍

You can see only:

A: before fetch
C: after fetch call

so far. The "B: fetch succeeded" log will appear later, when the network request finishes. That’s the async part doing its job.

Now let’s bring in the failure path with .catch() ✨

Idea in plain words:

.catch() runs when the Promise is rejected (for example, a network failure or an error thrown in .then()).

Simple mental picture:

.then(...) → what to do on success
.catch(...) → what to do on failure

Make sense so far if I say:

“.catch() is the error handler partner of .then()”?

Yes, that makes sense

Great ☑

Now let’s use .catch() with a request that will definitely fail.

In the code editor, replace your current code with this and run it:

Javascript
```
console.log("1: before bad fetch");

fetch("https://this-domain-should-not-exist-12345.com")
  .then(() => {
    console.log("2: this will NOT run on failure");
  })
  .catch(() => {
    console.log("3: fetch failed");
  });

console.log("4: after bad fetch call");
```
Then hit Submit code.

Again, don’t explain the output yet — just get it running.

We’ll talk about which logs appear and in what order right after.

Javascript
```
console.log("1: before bad fetch");

fetch("https://this-domain-should-not-exist-12345.com")
  .then(() => {
    console.log("2: this will NOT run on failure");
  })
  .catch(() => {
    console.log("3: fetch failed");
  });

console.log("4: after bad fetch call");
```
Perfect run

From your output:

1: before bad fetch
4: after bad fetch call
(and later, in a real network environment, we’d expect 3: fetch failed from .catch())

Key idea:

The console.log lines outside fetch (1 and 4) run immediately, in order.
The .catch(() => { console.log("3: fetch failed"); }) runs later, when the Promise is rejected because the domain is bad.

So .catch() is correctly set up as your failure handler ☑

Next concept: sometimes the network succeeds, but your own code inside .then() throws an error. That error should also be caught by .catch().

I’ll show that with a tiny example next. Ready for that?

Yes, show me

Nice

Step by step, here’s the new idea:

Sometimes the request succeeds, so .then() runs, but inside that .then() you have a bug, like:

Javascript
```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("In then before error");
    throw new Error("Something went wrong inside then");
  })
  .catch(() => {
    console.log("In catch after error");
  });
```
Flow:

fetch(...) succeeds → .then callback runs.
Inside .then, we throw new Error(...).
That thrown error makes the Promise reject, so .catch(...) runs.

So: errors thrown inside .then() are also caught by .catch() 🎯

Make sense so far?

If yes, I'll have you code a small version of this yourself in the editor.

yes akes sense

Great 😄

Now I want you to **trigger an error inside .then() and catch it with .catch() yourself.

In the code editor, replace your code with this (type it):

Javascript
```
console.log("A: start");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("B: inside then before error");
    throw new Error("Boom from then");
  })
  .catch(() => {
    console.log("C: inside catch after error");
  });

console.log("D: end of script");
```
Run it.
Then Submit code.

After that, we'll talk about which of A/B/C/D you expect to see, and in what order.

Javascript
```
console.log("A: start");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("B: inside then before error");
    throw new Error("Boom from then");
  })
  .catch(() => {
    console.log("C: inside catch after error");
  });

console.log("D: end of script");
```
Nice job running that 👏

From the output you saw:

A: start
D: end of script

The B and C logs happen later (after the async work), just like before.

What matters conceptually:

A logs immediately.
D logs immediately after setting up the fetch.
When the request succeeds, the .then callback runs:
B logs.
Then throw new Error("Boom from then"); happens.
That thrown error makes the Promise reject, so .catch runs:
C logs.

So the logical order in time is:

A, D, B, C ☑
Now I want to check that output-order idea more directly.

Without running any new code, which order do you think the console messages will appear in for this exact program?

You can just answer in text, like:

A, D, B, C or something else.

you just told me uness youre talking about somenew code a d b c

You're right — I was asking about this same code, not new code 😉

And your answer A, D, B, C is exactly the right order 🎯

A and D → run right away (synchronous)
B → runs later, when the fetch succeeds and .then runs
C → runs after the error is thrown inside .then, handled by .catch

So you've now seen:

.then() handling success.
.catch() handling a failed fetch.
.catch() also handling an error thrown inside .then.
And you correctly predicted async log order.
At this point, you've basically hit all the outcomes for this lesson.

Do you want to wrap up this lesson here and move on to the next one, or do one more quick practice on .then / .catch output order first?

Move to the next lesson

Done — this lesson is officially wrapped up ☑

Quick recap of what you can now do:

Use fetch(...).then(...) to handle a successful request (without touching the body).

Use .catch(...) to handle a failed request (like a bad domain).
Throw an error inside .then() and know it will be caught by the following .catch().
Predict the console log order when mixing normal logs, .then, and .catch — you correctly used the pattern: sync first, then async.

🔥 Solid async foundation, Ron.

Whenever you're ready, hit Next Lesson in the Learn tab and we'll keep building on this.

```
console.log("A: start");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(() => {
    console.log("B: inside then before error");
    throw new Error("Boom from then");
  })
  .catch(() => {
    console.log("C: inside catch after error");
  });

console.log("D: end of script");
```